Ders 13 Çalışma Özeti

Ders 13 Çalışma Özeti

Genel Konular

- Bu kayıt iki bölümden oluşur:
  - Laboratuvardan sonra yapılan ders: iteratif algoritmaların analizine devam (iç içe döngüler, `while` ile katlanarak artan döngüler, `if` içeren döngüler, toplam formülleri), global değişkenlerin neden kullanılmadığı ve rastgele sayılar konusuna giriş.
  - Aynı günün öğle saatinde yapılan ikinci laboratuvar: kurallar ve seyrek matrisin sıkıştırılması sorusu.
- Hoca, bu grubun diğer gruba göre bazı konularda geride olduğunu söyler. Bu yüzden laboratuvar günü de ders yapılır. Devam alınmaz.
- Ders başında bir öğrencinin polinom değeri hesaplayan kodu incelenir. `x = 30` gibi büyük değerlerde `printf` beklenmedik bir çıktı veriyordu. Kaynak derste bulunamadı ve ders sonrasına bırakıldı.
- T'nin değeri ile karmaşıklık
  - Hoca, iki şeyin karıştırılmaması gerektiğini tekrar vurgular. `T`'nin son değeri, toplam veya çarpım sembolünün içine yazılan ifadeye bağlıdır. Karmaşıklık ise temel işlemin kaç kez yapıldığıdır.
  - Örnek: `T = T + T` olan tek döngüde `T`'nin değeri 2^n'dir, ama döngü n kez döner ve karmaşıklık n'dir.
  - Bu derste kodlar C yerine MATLAB benzeri sözde kodla yazılmıştır.
- İç içe döngü: `for i=1:N`, `for j=1:i*i`
  - `T = T + 1`: iç döngü i^2 kez döner. $T = \sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$.
  - `T = T + i`: iç toplamda `i` sabittir, $i^2 \cdot i = i^3$. Ders sırasında bir öğrenci (Emircan) bu düzeltmeyi yaptı. $T = \sum_{i=1}^{n} i^3 = \left(\frac{n(n+1)}{2}\right)^2$.
  - `T = T + j`: iç toplam $\frac{i^2(i^2+1)}{2}$. $T = \frac{1}{2}\left(\sum i^4 + \sum i^2\right)$, burada $\sum_{i=1}^{n} i^4 = \frac{n^5}{5} + \frac{n^4}{2} + \frac{n^3}{3} - \frac{n}{30}$. Sınavda böyle bir formül gerekirse formül kağıdında verilir.
  - Bu döngünün karmaşıklığı (iterasyon sayısı) $\frac{n(n+1)(2n+1)}{6}$, yani n^3 mertebesindedir.
- `while` ile katlanarak artan döngü: `T=0; i=1; while (i<=N) T=T+1; i=i*2; end`
  - `i` 1, 2, 4, 8, ... diye gider, yani $2^0, 2^1, \ldots, 2^k$. $2^k = n$ ise $k = \log_2 n$. 0'dan başlandığı için terim sayısı $\log_2 n + 1$'dir.
  - `T = T + 1`: $T = \log_2 n + 1$. $n = 1024$ için 11.
  - `T = T + i`: eklenen değerler 1, 2, 4, ... yani $T = \sum_{i=0}^{\log_2 n} 2^i$. Bu üstel (exponential) bir toplamdır, $\sum i^2$ gibi polinom toplamlarla karıştırılmamalıdır.
  - Geometrik seri formülünün ispatı: $T = \sum_{i=0}^{n} a^i = 1 + a + \cdots + a^n$. $T - 1 = a(1 + a + \cdots + a^{n-1}) = a(T - a^n)$. Buradan $T(a-1) = a^{n+1} - 1$, yani $T = \frac{a^{n+1}-1}{a-1}$ ($a \neq 1$).
  - $a = 2$ ve üst sınır $\log_2 n$ konulursa $T = 2^{\log_2 n + 1} - 1 = 2n - 1$ bulunur. $2^{\log_2 n} = n$ olduğu kullanılır.
  - Karmaşıklık: iki durumda da döngü $\log_2 n + 1$ kez döner.
- $\sum_{i=0}^{n} i \cdot a^i$ formülü
  - Bir algoritmanın analizinde bu tür, doğrusal ve üstel çarpanı birlikte içeren bir toplam çıkabilir.
  - İspat: $T = 0 \cdot a^0 + 1 \cdot a + 2a^2 + \cdots + na^n$. Her iki taraf a ile çarpılıp $T - aT$ hesaplanır: $T(1-a) = a + a^2 + \cdots + a^n - na^{n+1}$. Sağdaki ilk kısım geometrik seridir ($\frac{a^{n+1}-1}{a-1} - 1$).

Sonuç: $T = \frac{1}{1-a}\left(\frac{a^{n+1}-1}{a-1} - 1 - na^{n+1}\right)$.

- `if` içeren döngü: `for i=1:N`, `for j=1:i*i`, `if mod(j,i)==0`, `T=T+1`
  - $i = 2$ için `j` 1-4 arasında gider, 2 ve 4 sayılır. $i = 3$ için 3, 6, 9, $i = 4$ için 4, 8, 12, 16. Yani her i için `T` i kadar artar.
  - `if` doğrudan toplam sembolüne yazılamaz. Desen bulunur ve öyle yazılır: $T = \sum_{i=1}^{n} i = \frac{n(n+1)}{2}$.
  - `T = T + 1` `else` kısmına yazılırsa: bütün durumlardan `if` durumunu çıkarmak yeterlidir. Bütün durumlar $\sum i^2$ olduğu için $T = \sum_{i=1}^{n}(i^2 - i)$.
- Katlanarak hesaplanan sınır: `K=1; for i=1:N K=K*2; end` ve ardından `for i=1:N`, `for j=1:K`, `T=T+1`
  - İlk döngüden sonra $K = 2^n$ olur (`K` her turda 2 ile çarpılır). Önce üst sınır bulunur, sonra toplam yazılır.
  - `T = T + 1`: $T = \sum_{i=1}^{n} 2^n = n \cdot 2^n$.
  - `T = T + i`: $T = 2^n \sum i = 2^n \cdot \frac{n(n+1)}{2} = 2^{n-1}n(n+1)$.
  - `T = T + j`: iç toplam $\frac{2^n(2^n+1)}{2}$ ve `i`'ye bağlı değil. $T = n \cdot \frac{2^n(2^n+1)}{2}$.
- Üç iç içe döngü: `for i=1:N`, `for j=1:i`, `for k=1:j`, `T=T+1`
  - En içteki toplam j, ortadaki $\sum_{j=1}^{i} j = \frac{i(i+1)}{2}$. $T = \frac{1}{2}\left(\sum i^2 + \sum i\right) = \frac{1}{2}\left(\frac{n(n+1)(2n+1)}{6} + \frac{n(n+1)}{2}\right)$.
  - Hoca, kendi kendinize çalışmak için içteki ifadeyi `i`, `j` veya `i + j` yaparak yeni sorular üretmeyi önerir. Fatih Hoca'nın sayfasında bu konuda çalışma sorusu yoktur.
- Basit sınırlar: `for i=1:N`, `for j=3:N`, `T=T+1`
  - İçeriye `j` değil 1 yazılır, çünkü `T` her turda 1 artar. İç döngüde $n-3+1 = n-2$ terim vardır.
  - $T = \sum_{i=1}^{n}(n-2) = n(n-2)$.
  - Karmaşıklık: temel işlem toplama, bir toplamaya 1 birim denir. Toplama $n(n-2)$ kez yapılır. Bir toplamanın süresi biliniyorsa programın süresi kabaca bu sayıyla çarpılarak tahmin edilir.
  - `T + 1` ile `T + j` arasında maliyet farkı yoktur. İkisi de iki tam sayının (32 bit) tek bir toplamasıdır. Biri kayan noktalı olsaydı iş değişirdi.
  - Yalnızca atamalar sayılırsa: `T = 0`, `i = 1`, n kez `j = 3` ve $n(n-2)$ kez `T = ...`. Bütün programı analiz etmek, her işlem türü için böyle ayrı ayrı sayıp toplamak demektir.
- Global değişkenler neden kullanılmaz?
  - İşletim sistemi birçok programı ve iş parçacığını (thread) eş zamanlı çalıştırır. Bir toplama işlemi makinede birkaç komuta (`MOV`, `ADD`) dönüşür. İşletim sistemi bu komutların arasında başka bir işe geçebilir.
  - İki iş aynı paylaşılan değişkene (ör. bir banka hesabına para yatırma ve çekme) eş zamanlı erişirse sonuç yanlış olabilir. Buna race condition denir. Çözümü semafor, monitör gibi karşılıklı dışlama (mutual exclusion) teknikleridir. Bunlar işletim sistemleri dersinde görülür.
  - Ayrıca parametre aktarımı (call by value, call by reference) işin mühendislik tarafıdır ve tercih edilen tasarım budur.
  - Bu derste yasak olanlar: global değişken, `switch` dışında `break`, `continue`, `goto`. `static` izne bağlıdır.
- Rastgele sayılara giriş: çember üzerinde desen çizme
  - Bir çember üzerinde m tane eşit aralıklı nokta üretilir (`P` matrisinde her noktanın `x` ve y koordinatı). Bir noktadan başlanır ve `yi = (i * s) mod m` ile bir sonraki noktaya geçilir, iki nokta arasına çizgi çekilir, sonra `i = yi` yapılır. s keyfi seçilmiş bir sabittir.
  - `i`, `s` ve `m` sabit olduğu için bir noktaya ikinci kez gelindiğinde aynı yol tekrarlanır (1, 3, 5, 7, 8, 1, 3, ...). İyi bir desen için bu tekrarın çok geç gelmesi gerekir.
  - Her noktaya kaç kez uğrandığı bir sayaç dizisinde tutulursa (`T[i]++`) dağılım görülebilir. Her noktaya eşit sayıda uğranıyorsa dağılım tek düzedir (uniform). Her noktanın olasılığı $\frac{1}{m}$'dir ve bir sonraki noktanın tahmin edilebilirliği en düşüktür. Dağılım düzgün değilse en sık gelen değeri tahmin etmek daha kolaydır.
  - s sabit yerine rastgele seçilirse tekrar etme olasılığı azalır.
- Rastgele sayı üreteçleri
  - Gerçek rastgele sayı üreteci (true random number generator) sayısal bilgisayarlarda yoktur. Örnek: hilesiz bir parayı veya zarı dış etkiler olmadan defalarca atmak.

  - Bilgisayarlar sözde rastgele sayı üreteci (pseudo random number generator) kullanır. Basit bir örnek doğrusal eşlik üretecidir: $r_n = (a \cdot r_{n-1} + c) \bmod m$. Derste verilen değerler $a = 1366$, $c = 150889$, $m = 714025$. Başlangıç değeri r_0 verilmelidir.
  - Dizi bir yerde mutlaka kendini tekrar eder. Tekrara kadar geçen uzunluk ne kadar büyükse üreteç o kadar iyidir. m büyüdükçe "rastgelelik illüzyonu" artar, döngü görülemez.
  - ANSI C örneği: $s_0 = 12345$ (seed, çekirdek), $s_{i+1} = (1103515245 \cdot s_i + 12345) \bmod 2^{31}$.
  - Seed değiştirilmezse program her çalıştığında aynı diziyi üretir. `srand(sabit)` de her çalıştırmada aynı diziyi verir.
  - `srand(time(NULL))`: `time(NULL)` 1 Ocak 1970'ten o ana kadar geçen saniyeyi verir. Bu değer her çalıştırmada farklı olduğu için seed de farklı olur.
  - `srand` `main`'in başında bir kez çağrılır. `rand` istenildiği kadar çağrılır. `srand`'ı döngüye koymak yaygın bir hatadır ve gereksizdir.
  - Ertesi gün: zar, 52 kart, Gauss dağılımı, robot örneği ve pi sayısının bulunması.
- Ders sonrası sorular
  - Bir öğrenci: `for` koşulunda `j = n-i-1` yazılmış, `expression is not assignable` hatası ve hep aynı çıktı. Koşulda `==` gerekir. Atama, 0 dışında her değerde doğru sayılır. Ayrıca `exchange` değişkeni 0 ile başlatılıp `while` içinde 1 yapılmış. Ya 1 ile başlatılmalı ya da `do-while` kullanılmalı.
  - Hoca `do-while`'ı önerir: koşul ilk turda kontrol edilmeden bir kez çalışır, ikili aramada ve giriş (login) işlemlerinde aynı kodu iki kez yazmayı önler. MATLAB'da `do-while` yoktur.
  - Tarık: seyrek matris laboratuvar sorusunda büyük satır sayısında program çöküyordu. Dev-C++'ta debug için dosyanın proje olarak açılması gerekti. Hoca, matrisi `MAX`'a göre değil gerçek satır ve sütun sayısına göre yazdırmasını, döngü sınırlarını kontrol etmesini ve boş `else` bırakmamasını söyler. Hata büyük olasılıkla matrisin dışına taşmadır.
  - Bir öğrencinin insertion sort'u yanlış sıralıyordu. Insertion sort ikinci elemandan (`i = 1`) başlar.
  - Bir öğrenciye Dev-C++'ta debug kullanımı gösterilir: breakpoint, `Next line`, `Into function`, değişkenin üzerine gelme ve `Add watch`. Değişiklikten sonra debug'dan önce derlemek gerekir.
- Laboratuvar 2 (aynı gün, asistanlar tarafından)
  - Kurallar: dosya adı `Lab2_ogrenciNo.c` biçiminde olacak, öğrenci adı kodda yazacak. C++ dosyası gönderilmeyecek. `for` içinde değişken tanımlanmayacak, bütün tanımlar fonksiyonun başında olacak. Süre 85 dakika (80 + 5 dakika yükleme). Formdan sonra e-postayla gönderilen ödevler kabul edilmeyecek. Forma sık kullanılan bir e-posta adresi yazılmalı.
  - Kendi fonksiyonlarınızı yazabilirsiniz. Ama `strcmp` gibi iki `for` ile kendinizin yazabileceği işler için hazır kütüphane fonksiyonu kullanılmamalıdır.
  - Soru: sıfırı çok olan (seyrek) bir A matrisini, her satırında sıfırdan farklı bir elemanın (satır, sütun, değer) bilgisi olan 3 sütunlu bir matrise sıkıştırmak. İkinci aşamada sıkıştırılmış matristen A'yı geri oluşturmak. Bir menü: 1 sıkıştır, 2 geri oluştur, 3 çıkış. Her işlemden sonra menü yeniden gelir.
  - Matris kare olmak zorunda değildir, boyutlar kullanıcıdan alınır. Satır başına birden fazla sıfırdan farklı eleman olabilir. Bütün elemanlar sıfırsa sıkıştırılmış matris boş olur.
  - Asistan, menü hazırlamanın veri yapıları ve algoritma analizi derslerindeki ödevler için iyi bir hazırlık olduğunu söyler.

Hocanın Özellikle Vurguladığı Kısımlar

- T'nin son değeri ile algoritmanın karmaşıklığını karıştırmayın.
- Toplam sembolünü yazmadan önce sınırları bulun: döngü değişkeni nasıl artıyor, kaç terim var? Katlanarak artan döngüde terim sayısı $\log_2 n + 1$'dir.
- $\sum i^2$ gibi polinom toplamlar ile $\sum 2^i$ gibi üstel toplamlar farklıdır. Bu hata çok yapılıyor.
- `if` içeren döngülerde birkaç değer için deseni çıkarın. `else` durumunu "bütün durumlar eksi `if` durumu" olarak bulun.

- Global değişken kullanmayın. `srand`'ı bir kez çağırın.
- Laboratuvarda C dosyası gönderin ve dosya adlandırma kurallarına uyun.

Kısa Tekrar Notları

- $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$, $\sum i^2 = \frac{n(n+1)(2n+1)}{6}$, $\sum i^3 = \left(\frac{n(n+1)}{2}\right)^2$.
- $\sum_{i=0}^{n} a^i = \frac{a^{n+1}-1}{a-1}$. $\sum_{i=0}^{\log_2 n} 2^i = 2n-1$ (n 2'nin kuvveti ise).
- `i = i * 2` ile `n`'e kadar: $\log_2 n + 1$ tur.
- `mod(j, i) == 0`, `j` 1'den i^2'ye: her `i` için i tane.
- `K` döngüde n kez ikiyle çarpılırsa $K = 2^n$.
- Doğrusal eşlik üreteci: $r_n = (ar_{n-1} + c) \bmod m$. `srand(time(NULL))` bir kez.

Detaylı Açıklamalar

Formüllerin sayısal kontrolü

Derste bulunan bütün sonuçlar ders sonrasında döngüler gerçekten çalıştırılarak $n = 10$ için kontrol edildi. `j` 1'den i^2'ye giden döngü için `T + 1`, `T + i` ve `T + j` sırasıyla 385, 3025 ve 12859 verdi. Formüllerle aynıdır. `mod` döngüsü `if` kolunda 55, `else` kolunda 330 verdi. `K` döngüsü 10240, 56320 ve 5248000 verdi. Üçlü döngü 220, `j = 3:N` döngüsü 80 verdi. $\sum_{i=0}^{n} ia^i$ formülü $a = 3$, $n = 6$ için 6015 ile doğrulandı.

Ders sonrası notlar

- $2n - 1$ **sonucu:** $\sum_{i=0}^{\log_2 n} 2^i = 2n - 1$ yalnızca n 2'nin tam kuvvetiyse doğrudur. Döngü $i \le n$ olduğu sürece döner, yani son terim n'i aşmayan en büyük 2'nin kuvvetidir. Genel sonuç $2^{\lfloor \log_2 n \rfloor + 1} - 1$'dir. $n = 1000$ için `T` 1023 olur, 1999 değil. Tur sayısı da genel olarak $\lfloor \log_2 n \rfloor + 1$'dir ($n = 1000$ için 10).
- **`K` döngüsünde `T + i`:** Derste sonuç önce $2^n \cdot n(n+1)$ olarak söylendi, sonra $2^{n-1}n(n+1)$ de olur denildi. Doğrusu $2^n \cdot \frac{n(n+1)}{2} = 2^{n-1}n(n+1)$'dir. İlk ifade iki kat büyüktür.
- **`j = 3:N` döngüsü:** Derste bir an "$n^2 - 2$" dendi. Doğrusu $n(n-2) = n^2 - 2n$'dir. Atama sayımında `T = 0`, `i = 1`, n kez `j = 3` ve $n(n-2)$ kez `T = T + 1` vardır. Toplam $n^2 - n + 2$ bulunur. Derste $n^2 - n + 3$ söylendi. Döngü artırmaları da atama sayılırsa sonuç yine büyür. Mertebe her durumda n^2'dir.
- **Süre tahmini:** Hocanın örneğinde bir toplama 0,0001 saniye ve $n = 10^6$ alındı. $10^6 \cdot (10^6 - 2) \cdot 10^{-4} \approx 10^8$ saniyedir. Bu yaklaşık 3,2 yıl eder. n^2 algoritmaların büyük girdilerde neden kullanılamadığını iyi gösterir.
- **`mod` örneğindeki kalan:** Derste "1'in 1'e bölümünden kalan 1'dir" dendi. 1'in 1'e bölümünden kalan 0'dır. Bu yüzden $i = 1$, $j = 1$ durumu sayılır ve $i = 1$ için `T` 1 artar. Sonuç değişmez.
- **"i^k toplamının genel formülü yok" ifadesi:** $\sum_{i=1}^{n} i^k$ için genel bir formül vardır (Faulhaber formülü, Bernoulli sayılarıyla yazılır). Ama her k için açık hali farklı ve uzundur. Bu yüzden ders kapsamında $k = 1, 2, 3, 4$ için formüller kullanılır.
- **Çemberde desen ve `(i * s) mod m`:** 1'den başlayan bu çarpımsal kural bütün noktaları gezemez. 0 noktasına hiç gelinmez ve m ile ortak böleni olan noktalara da ulaşılamaz. Ders sonrasındaki denemelerde $m = 10$, $s = 3$ için yalnızca 1, 3, 9, 7 noktaları gezildi. $m = 12$, $s = 5$ için yalnızca 1 ve 5 gezildi. Bütün noktaları gezmek için toplamalı bir kural (`yi = (i + s) mod m`, $\gcd(s, m) = 1$) kullanılabilir. Bu kural m adımda bütün noktaları gezer. Doğrusal eşlik üreteci iki işlemi birleştirir: $(a \cdot r + c) \bmod m$.
- **`time` ve kaynak kodu:** Hoca `time`'ın bir makro olabileceğini söyledi. `time` `<time.h>`'de tanımlı standart bir fonksiyondur. `rand` ve `srand` `<stdlib.h>`'tedir, `<stdio.h>`'de değil.

Standart kütüphanenin gerçek kaynak kodları (ör. GNU C Library) açık kaynaktır ve incelenebilir.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.